UBSV (Unbounded Bitcoin Satoshi Vision) - Architecture Overview

Index

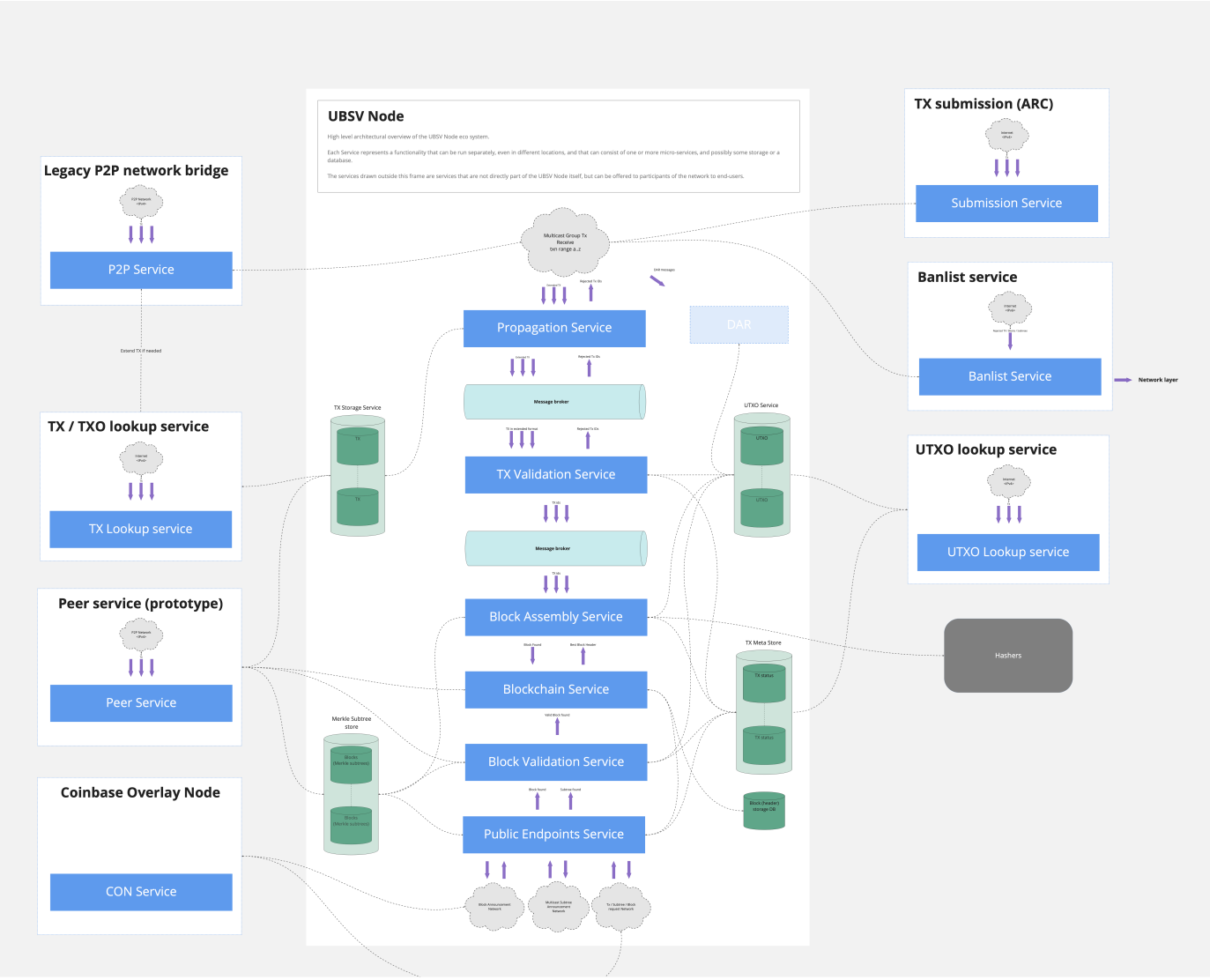
1. Overview

In the early stages of Bitcoin's development, a block size limit of 1 megabyte per block was introduced as a temporary measure. This limit effectively restricts the network's capacity to approximately 3.3 to 7 transactions per second. As Bitcoin's adoption has expanded, this constraint has increasingly led to transaction processing bottlenecks, causing delays and higher transaction fees. These issues have highlighted the critical need for scalable solutions within the Bitcoin network.

UBSV (Unbounded Bitcoin Satoshi Vision) is a major component of the Teranode Project, being developed by the BSV Association. UBSV addresses the challenges of vertical scaling by instead spreading the workload across multiple machines. This horizontal scaling approach, coupled with an unbound block size, enables network capacity to grow with increasing demand through the addition of cluster nodes, allowing for Bitcoin scaling to be truly unbounded.

UBSV provides a robust node processing system for Bitcoin that can consistently handle over **1M transactions per second**, while strictly adhering to the Bitcoin whitepaper. The node has been designed as a collection of services that work together to provide a decentralized, scalable, and secure blockchain network. The node is designed to be modular, allowing for easy integration of new services and features.

The diagram below shows all the different microservices, together with their interactions, that make up the UBSV node.



Various services and components are outlined to show their interactions and functions within the system:

1. **UBSV Node Core Services**:

- **Propagation Service**: Receives new transactions and sends them to the validator service.
- **TX Validation Service**: Checks transactions for correctness and adherence to network rules.
- **Block Assembly Service**: Assembles blocks to be included blockchain.
- **Blockchain Service**: Manages the block headers and list of subtrees in a block.
- **Block Validation Service**: Validates new subtrees and blocks before they are added to the blockchain.

2. **Overlay Services**:

- **Legacy P2P Network Bridge**: This service handles the legacy peer-to-peer communications within the blockchain network. As older (legacy) nodes will not be able to directly communicate with the newer (UBSV) nodes, this service acts as a bridge between the two types of nodes.
- **Peer Service**: This service handles node discovery, managing connections with peer nodes in the network.
- **Coinbase Overlay Node**: This service tracks and stores the Coinbase transactions, which are the first transactions in a block that create new coins and reward miners.

- **TX / TXO Lookup Service**: Queries "in scope" transaction and transaction ouput data.
- **TX Submission (ARC)**: Manages the submission of transactions to the network on behalf of the propagation service.
- **Banlist Service**: Maintains and verifies against a list of banned entities or nodes.
- **UTXO Lookup Service**:
    - **UTXO Lookup Service**: Retrieves information about **unspent** transaction outputs, which are essential for validating new transactions.

3. **Other Components**:

- **Message Broker**: A middleware that facilitates communication between different services.
- **TX Meta Store**: Keeps track of the status of transactions within the system.
- **Hashers**: Perform the computational work of hashing that is central to the mining process.

2. Data Model and Propagation

The UBSV data model addresses scalability and efficiency issues found in the BTC design by introducing data abstractions that make propagation across the network more optimal. This section provides a summary of the key points and how they represent improvements over the BTC design:

2.1. Block Size

- **Bitcoin**: Fixed at 1MB, limiting the number of transactions per block.
- **BSV**: Increased to 4GB, allowing significantly more transactions per block.
- **UBSV**: Unbounded block size, enabling potentially unlimited transactions per block, increasing throughput, and reducing transaction fees.

2.2. Bitcoin Data Model

A comparison of the BTC and UBSV Data Model helps to understand how the latter improves on the former.

The Bitcoin data model is as follows:

2.2.1. Transactions

Transactions are broadcast and included in blocks as they are found.

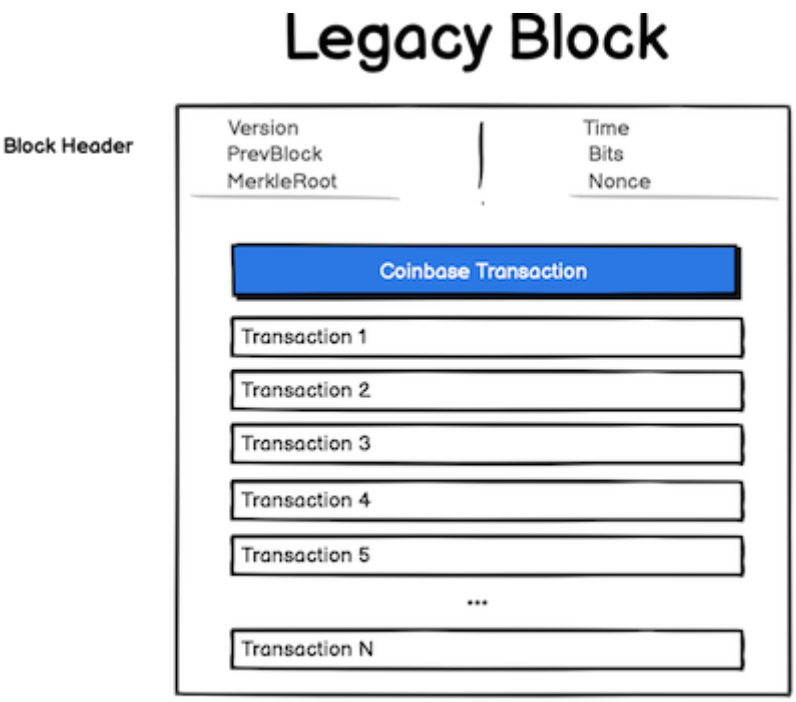
Current Transaction format:

Field	Description	Size
Version no	currently 2	4 bytes
In-counter	positive integer VI = [[VarInt]]	1 - 9 bytes

Field	Description	Size
list of inputs	Transaction Input Structure	<in-counter> qty with variable length per input
Out-counter	positive integer VI = [[VarInt]]	1 - 9 bytes
list of outputs	Transaction Output Structure	<out-counter> qty with variable length per output
nLocktime	if non-zero and sequence numbers are < 0xFFFFFFFF: block height or timestamp when transaction is final	4 bytes

2.2.2. Blocks:

Blocks contain all transaction data for the transactions included.



Note how the Bitcoin block contains all transactions (including ALL transaction data) for each transaction it contains, not just the transaction Id. This means that the block size would be very large if many transactions were included. At scale, this is not practical, as the block size would be too large to propagate across the network in a timely manner.

Let's see next how the UBSV data model addresses these issues.

2.3. UBSV Data Model

2.3.1. Transactions:

UBSV Transactions (referred to as "Extended Transactions") include additional metadata to facilitate processing, and are broadcast to nodes as they occur.

UBSV is expected to receive new transactions using the extended format only, with the legacy format not supported. A wallet will be required to create new transactions in extended format in order to communicate with UBSV.

The Extended Format adds a marker to the transaction format:

Field	Description	Size
Version no	currently 2	4 bytes
EF marker	**marker for extended format**	**0000000000EF**
In-counter	positive integer VI = [[VarInt]]	1 - 9 bytes
list of inputs	**Extended Format** transaction Input Structure	<in-counter> qty with variable length per input
Out-counter	positive integer VI = [[VarInt]]	1 - 9 bytes
list of outputs	Transaction Output Structure	<out-counter> qty with variable length per output
nLocktime	if non-zero and sequence numbers are < 0xFFFFFFFF: block height or timestamp when transaction is final	4 bytes

The Extended Format marker allows a library that supports the format to recognize that it is dealing with a transaction in extended format, while a library that does not support extended format will read the transaction as having 0 inputs, 0 outputs and a future nLock time. This has been done to minimize the possible problems a legacy library will have when reading the extended format. It can in no way be recognized as a valid transaction.

The input structure is the only additional thing that is changed in the Extended Format. The original input structure looks like this:

Field	Description	Size
Previous Transaction hash	TXID of the transaction the output was created in	32 bytes
Previous Txout-index	Index of the output (Non negative integer)	4 bytes
Txin-script length	Non negative integer VI = VarInt	1 - 9 bytes

Field	Description	Size
Txin-script / scriptSig	Script	<in-script length>-many bytes
Sequence_no	Used to iterate inputs inside a payment channel. Input is final when nSequence = 0xFFFFFFFF	4 bytes

In the Extended Format, we extend the input structure to include the previous locking script and satoshi outputs:

Field	Description	Size
Previous Transaction hash	TXID of the transaction the output was created in	32 bytes
Previous Txout-index	Index of the output (Non negative integer)	4 bytes
Txin-script length	Non negative integer VI = VarInt	1 - 9 bytes
Txin-script / scriptSig	Script	<in-script length>-many bytes
Sequence_no	Used to iterate inputs inside a payment channel. Input is final when nSequence = 0xFFFFFFFF	4 bytes
Previous TX satoshi output	**Output value in satoshis of previous input**	**8 bytes**
Previous TX script length	**Non negative integer VI = VarInt**	**1 - 9 bytes**
Previous TX locking script	**Script**	**<script length>-many bytes**

The Extended Format is not backwards compatible, but has been designed in such a way that existing software should not read a transaction in Extend Format as a valid (partial) transaction. The Extended Format header (0000000000EF) will be read as an empty transaction with a future nLock time in a library that does not support the Extended Format.

To know more about the Extended Transaction Format, please refer to the Bitcoin Improvement Proposal 239 (09 November 2022).

2.3.2. Subtrees:

The Subtrees are an innovation aimed at improving scalability and real-time processing capabilities of the blockchain system.

Unique to UBSV: The concept of subtrees is a distinct feature not found in the BTC design.

1. A subtree acts as an intermediate data structure to hold batches of transaction IDs (including metadata) and their corresponding Merkle root.
    - Note that the size of the subtree could be any number of transactions, just as long as it is a power of 2 (16, 32, 64...). The only requirement is that all subtrees in a block have to be the same size. At peak throughput, subtrees will contain millions of transaction Ids.

2. Each subtree computes its own Merkle root, which is a single hash representing the entire set of transactions within that subtree.

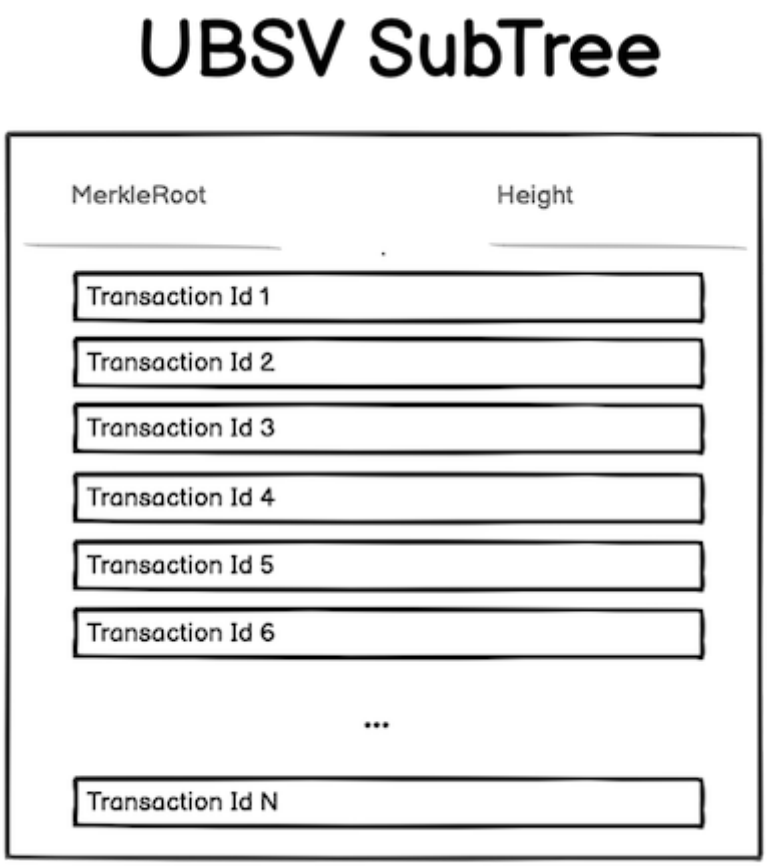
Efficiency: Subtrees are broadcast every second (assuming a baseline throughput of 1M transactions per second), making data propagation more continuous rather than batched every 10 minutes. Notice that blocks are still created every 10 minutes, but the subtrees are broadcast every second.

1. By broadcasting these subtrees at such a high frequency, receiving nodes can validate these batches quickly and continuously, having them "pre-approved" for inclusion in a block.
2. This contrasts with the BTC design, where a new block, and hence a new batch of transactions, is broadcast approximately every ten minutes after being confirmed by miners.

Lightweight: Subtrees only include transaction IDs, not the full transaction data, since all nodes already have the transactions, thus reducing the size of the data to propagate.

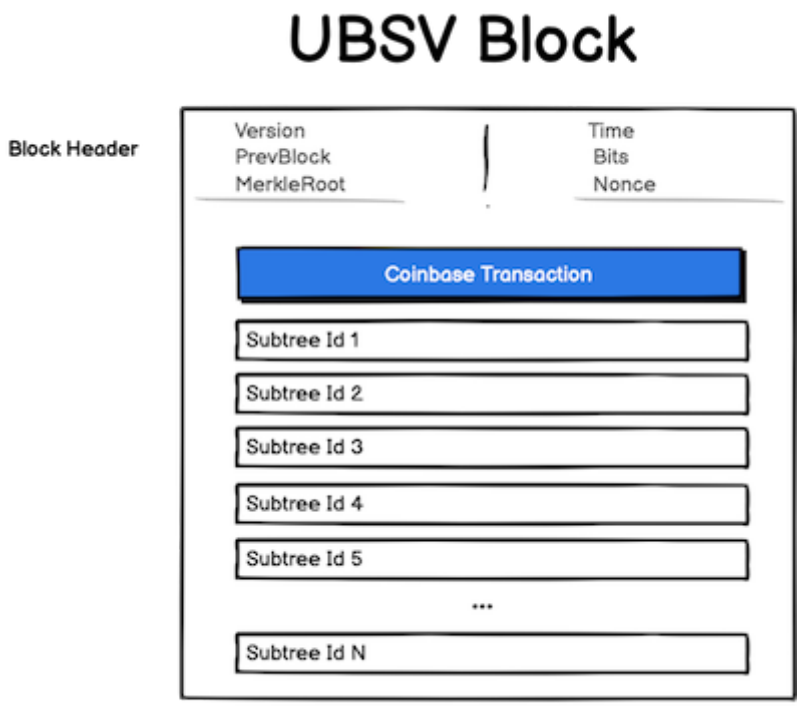
1. Since all nodes participating in the network are assumed to already have the full transaction data (which they receive and store as transactions are created and spread through the network), it's unnecessary to rebroadcast the full details with every subtree.

2. The subtree then allows nodes to confirm they have all the relevant transactions and to update their state accordingly without having to process vast amounts of data repeatedly.



2.3.3. Blocks:

Blocks contain lists of subtree identifiers, not transactions. This is practical for nodes because they have been processing subtrees continuously, this allows for quick validation of blocks.



Please note that the use of subtrees within blocks represents a data abstraction for a more optimal propagation of transactions. The data model is still the same as Bitcoin, with blocks containing transactions. The subtrees are used to optimize the propagation of transactions and blocks.

2.4. Advantages of the UBSV Model

- **Faster Validation**: Since nodes process subtrees continuously, validating a block is quicker because it involves validating the presence and correctness of subtree identifiers rather than individual transactions.
- **Scalability**: The model supports a much higher transaction throughput (> 1M transactions per second).

2.5. Network Behavior

- **Transactions**: They are broadcast network-wide, and each node further propagates the transactions.
- **Subtrees**: Nodes broadcast subtrees to indicate prepared batches of transactions for block inclusion, allowing other nodes to perform preliminary validations.
- **Block Propagation**: When a block is found, its validation is expedited due to the continuous processing of subtrees. If a node encounters a subtree within a new block that it is unaware of, it can request the details from the node that submitted the block.

This proactive approach with subtrees enables the network to handle a significantly higher volume of transactions while maintaining quick validation times. It also allows nodes to utilize their processing power more evenly over time, rather than experiencing idle times between blocks. This model ensures that the UBSV

can scale effectively to meet high transaction demands without the bottlenecks experienced by the BTC network.

3. Node Workflow

At a high level, the UBSV node performs the following functions:

1. **Transaction Submission**: UBSV nodes are subscribed to a IPv6 or alternative broadcast service, and transactions are expected to be received by all nodes.

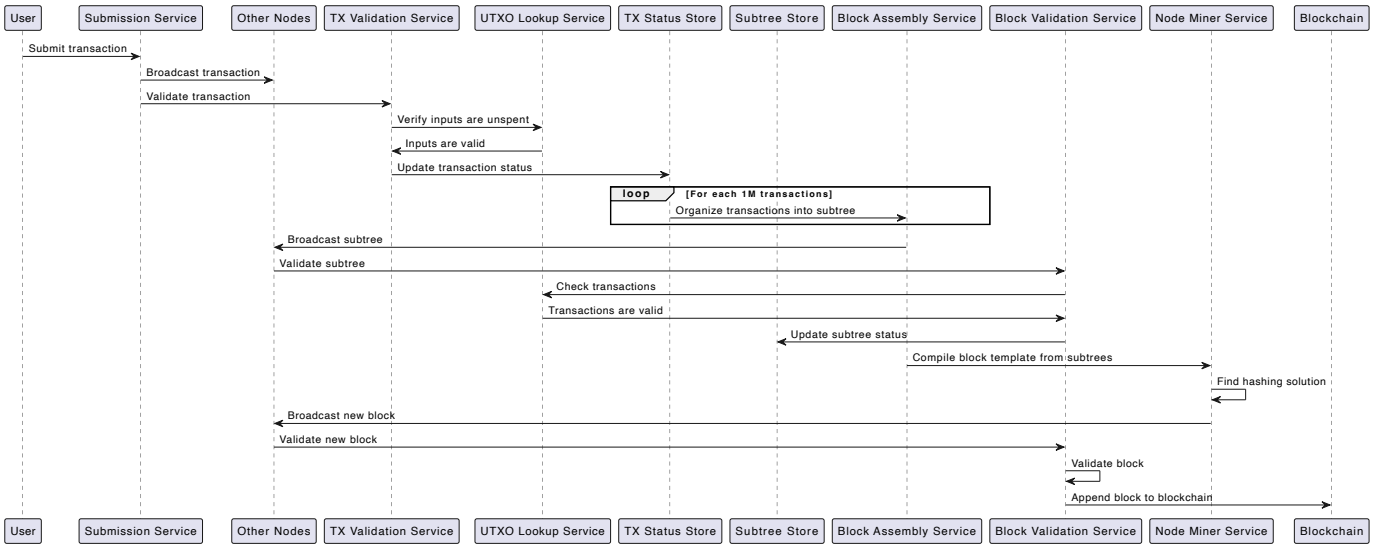
2. **Transaction Validation**: Transactions are validated by the TX Validation Service. This service checks each received transaction against the network's rules, ensuring they are correctly formed and that their inputs are valid and unspent (verified by the UTXO Lookup Service). Once validated, the status of transactions are updated in the TX Meta Store, indicating they have not been included in a block yet and are eligible for inclusion.

3. **Subtree Assembly**: The Block Assembly Service ingests transactions and organizes them into subtrees. "Subtrees" are a key component of the UBSV node, allowing for efficient processing of transactions and blocks. A subtree can contain up to 1M transactions. Once a subtree is created, it is broadcast to all other nodes in the network.
    - Note - nodes are expected to arrive to similar or equal subtree compositions. All nodes should have the same transactions in their subtrees, but the order of the transactions may differ. As they build their subtrees, nodes will broadcast these subtrees to each other.

4. **Subtree Validation**: The Block Validation Service validates the subtrees it receives. This involves checking the transactions within the subtree are known and eligible for inclusion in a subtree. Once validated, the status of the subtree is updated, marking it as eligible for inclusion in a block.
    - Note - If a subtree is not valid, it is discarded and not included in the block. If a subtree is valid, it will be stored in the Subtree Store, and later used by both block validation and block assembly.

5. **Block Assembly**: The Block Assembly Service compiles block templates consisting of subtrees. These templates are pre-blocks that the node miner service will use to create a full block. Once a hashing solution has been found, the block is broadcast to all other nodes for validation.

6. **Block Validation**: Once a node finds a valid hashing solution (a successful proof-of-work), the found block is sent to the Block Validation Service. This service checks the new block against the network's consensus rules. If the block is valid, the node will append it to the blockchain. Each node maintains a local copy of the blockchain, which is updated as new blocks are added.



4. Services

The node has been designed as a collection of microservices, each handling specific functionalities of the Bitcoin network.

4.1. Propagation Service

The Propagation service is responsible for receiving transactions from other nodes and forwarding them to the Validation service.

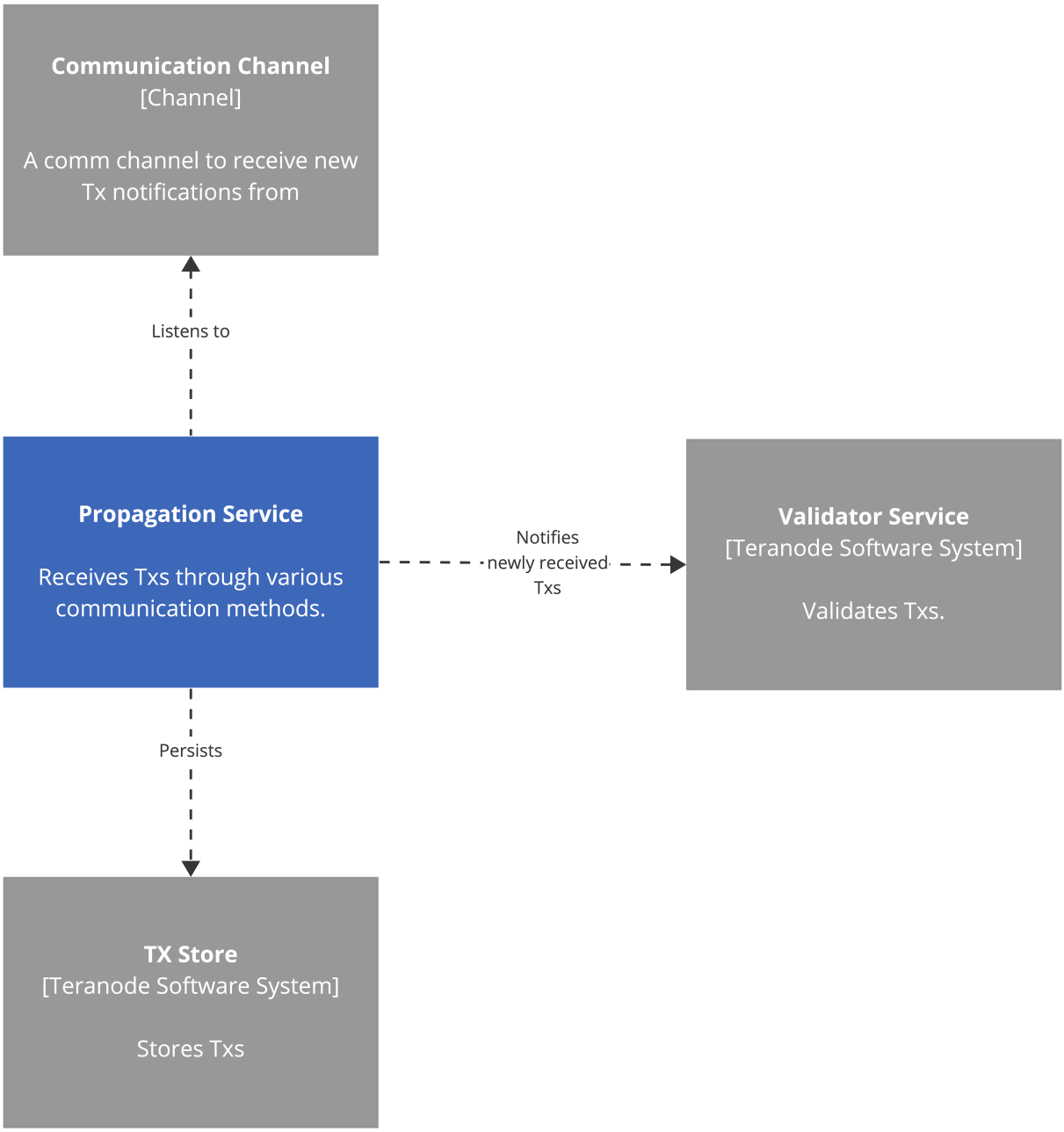
Here is a breakdown of the components as shown:

1. **TX Storage Service:** This is a datastore that holds all received transactions, either invalid or eligible for inclusion in the next block template.

2. **Multicast Receiver Service (Multiple Instances):** These services are responsible for listening to transactions that are being broadcast over the network. These services listen on IPV6 multicast network addresses reserved for Bitcoin transactions. There are multiple instances, each listening to a set of fixed IPV6 addresses, offering a horizontally scalable design that allows for handling more transactions by increasing the number of service instances. There can be an arbitrary number of these multicast receiver services operating, which is part of how the system achieves scalability.

3. **Message Broker:** This is the communication layer that the multicast receiver services use to forward the extended transactions to the Validation service. The use of a message broker introduces decoupling between the services, allowing for more scalable and maintainable systems.

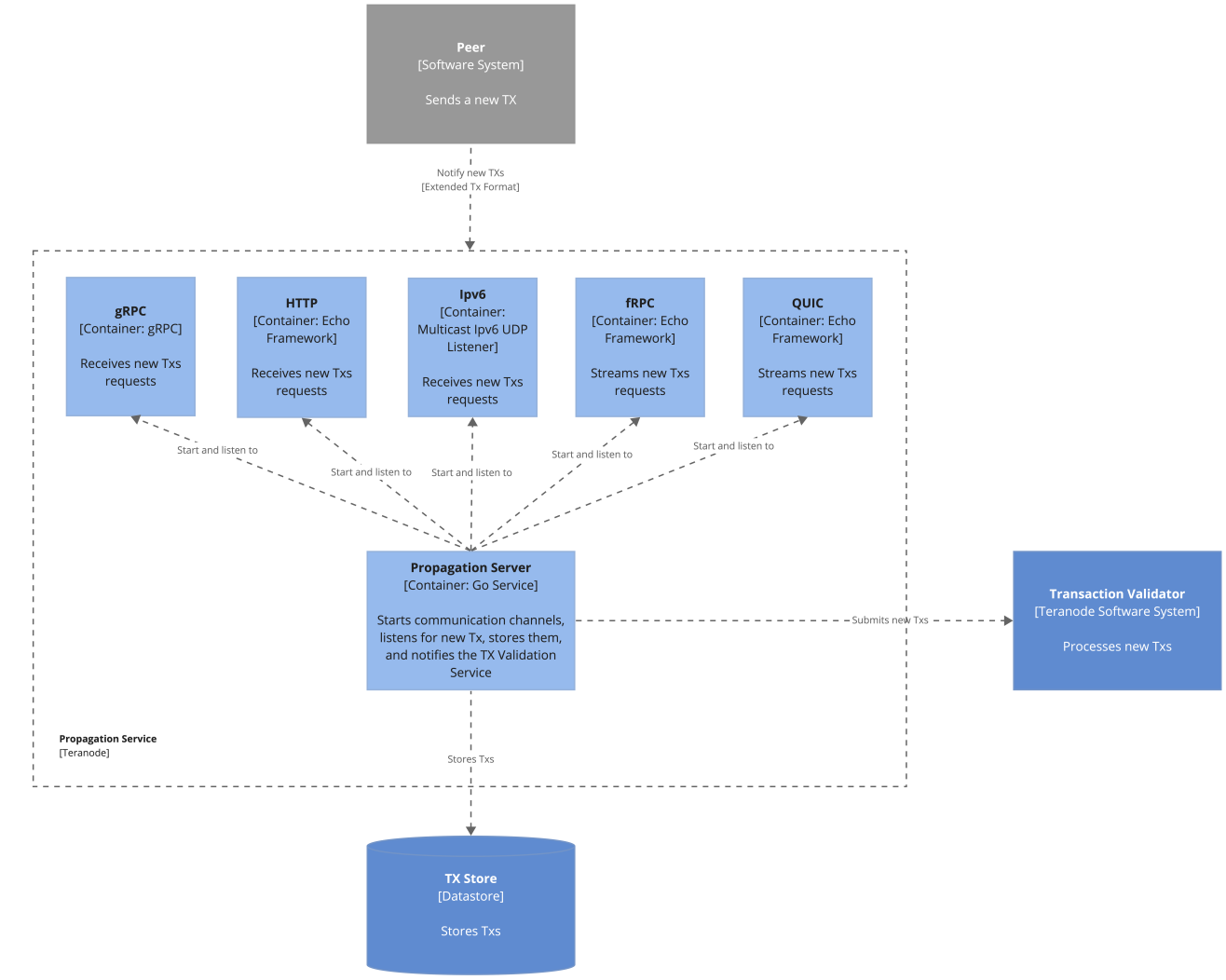
4. **Sanity Checks**: Before sending transactions on to the Validation Service, the Multicast Receiver Services perform basic validations to ensure transactions are correct and adhere to network protocols.



System Container Diagram for the Propagation Service

The system container diagram for the Propagation Service

Last modified: 16-Jan-2024



Note:

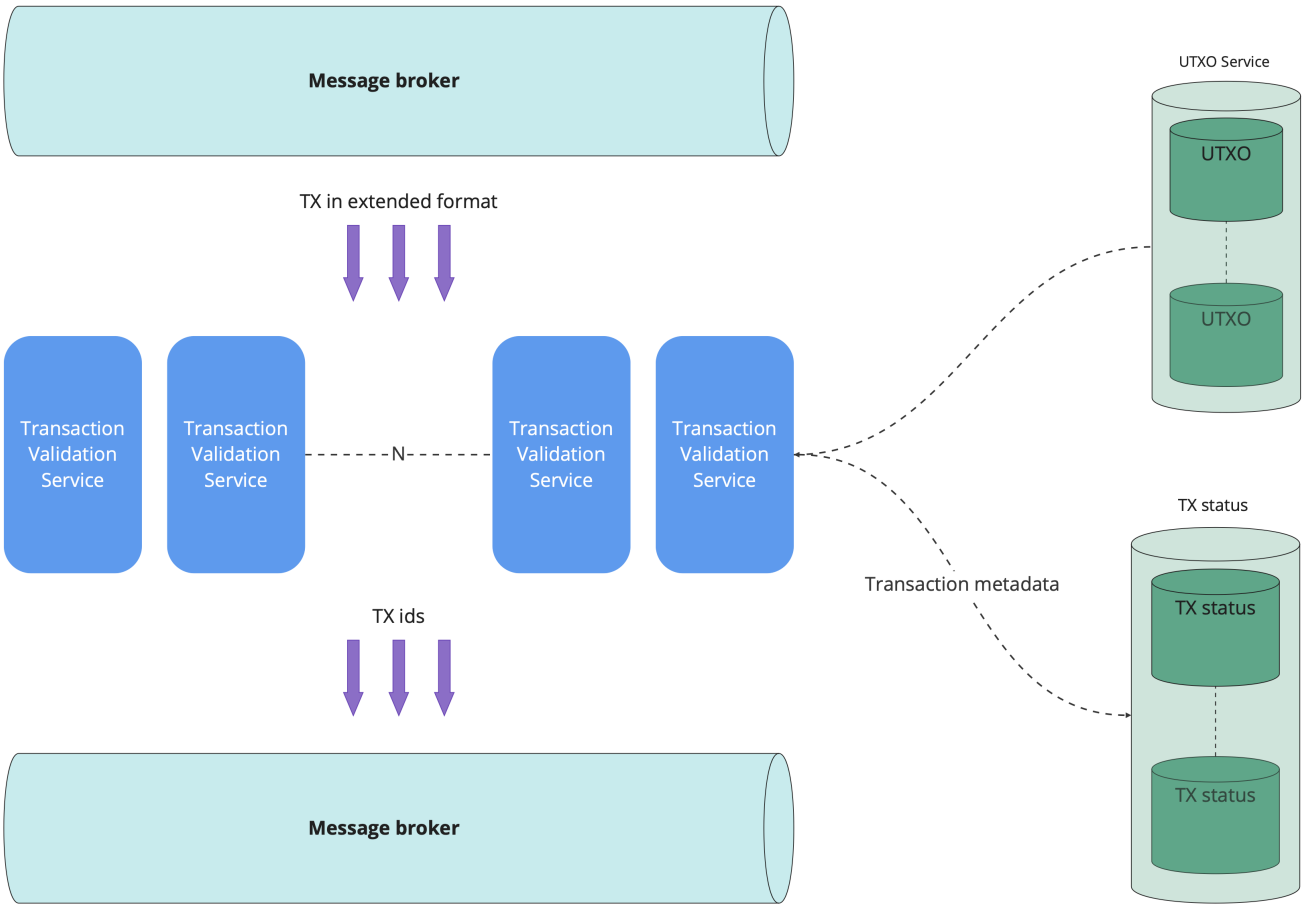
- **Communication via gRPC**: While the main network might use multicast for propagation, the test network simplifies operations by using gRPC, a high-performance, open-source universal RPC framework, for direct communication between the Propagation and Validation Services, bypassing the need for IPV6 broadcasting.

4.2. Transaction Validation

The TX Validation Service is responsible for validating a transaction according to the rules of the Bitcoin network and then sending the approved transaction ID forwards to block assembly.

The transactions that have passed the TX Validation Service are immediately marked as spent in the UTXO ("Unspent Transaction Output (UTXO)" store.

After the UTXOs have been marked as spent, the transaction metadata is stored in the TX Meta store and sent onwards to the Block Assembly Service via a Message Broker.



Here is a breakdown of the components as shown:

1. **Message Broker:** Communication layers that facilitate message passing between the Propagation Service and the Validation Service, and between the Validation Service and the Block Assembly.
2. **TX Validation Service (Multiple Instances):** Multiple instances of the service can be initiated, allowing to validate transactions in parallel, which will help to increase throughput and scalability.
4. **TX ids:** After validation, transaction identifiers (ids) are passed to the Block Assembly service.
5. **UTXO Service:** Datastore of UTXOs (the outputs from transactions that have not been spent and can be used as inputs in new transactions).
6. **TX Meta Store:** Datastore managing the statuses of transactions. If transactions are validated and not yet mined, they will be eligible for inclusion in a block.

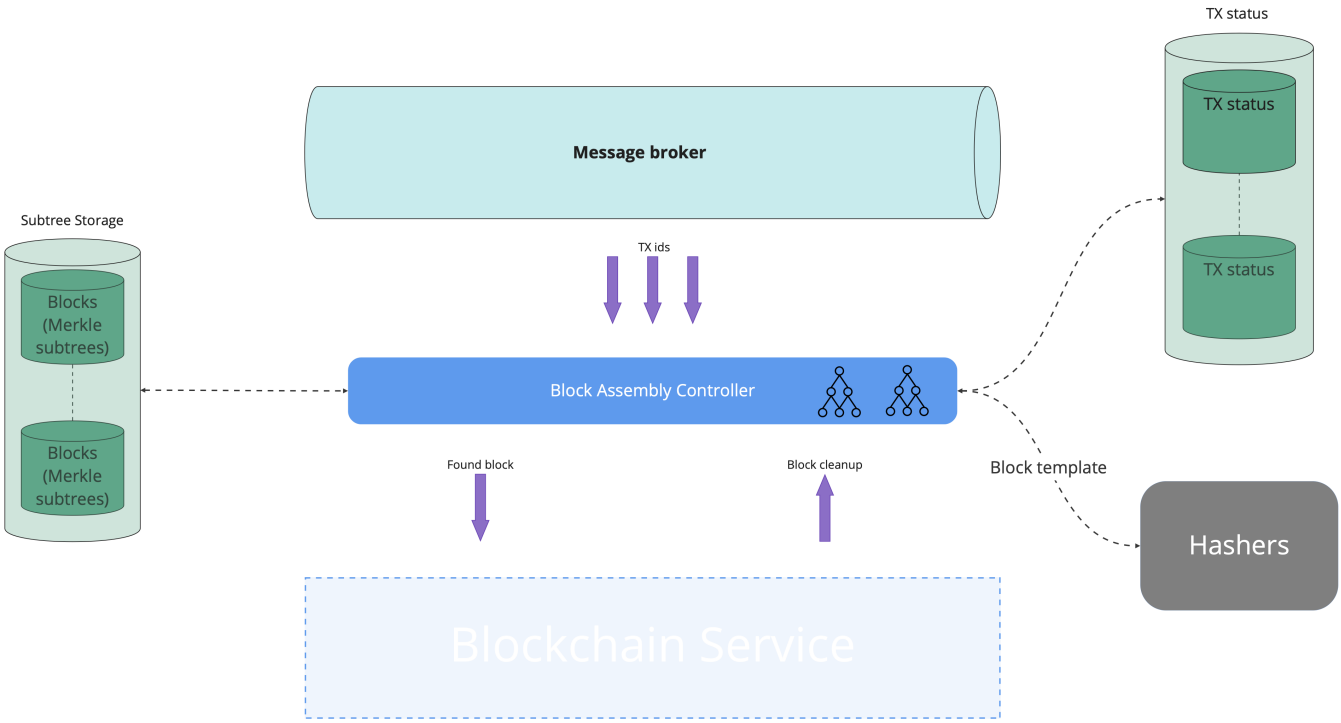
4.3. Block Assembly Service

This service is responsible for creating subtrees, as well as creating mining candidates (blocks) for Miner Services to hash against. The Block Assembly Service will broadcast any newly created subtrees and blocks to the network.

There are two distinct processes that the Block Assembly Service performs:

1. **Subtree Assembly**: The Block Assembly Service ingests transactions and organizes them into subtrees. As discussed in the UBSV Data Model section, subtrees are a key component of the UBSV node, allowing for efficient processing of transactions and blocks. A UBSV subtree is designed to contain at least 1M transactions. Once a subtree is created, it is broadcast to all other nodes in the network.

2. **Block Assembly**: The Block Assembly Service compiles block templates consisting of subtrees. Once a hashing solution has been found, the block is broadcast to all other nodes for validation.



The relevant components and their interactions are described below:

1. **Subtree Storage**: This stores blocks and their corresponding Merkle subtrees.

2. **Message Broker**: This is a middleware that allows for the decoupling of different parts of the blockchain system, facilitating asynchronous communication. Transaction IDs (Tx IDs) are sent from the Message Broker to the Block Assembly Controller.

3. **Block Assembly Controller**: This component orchestrates the process of creating new subtrees and new blocks. It receives transaction IDs from the Message Broker, performs necessary checks, and assembles subtrees and blocks.

4. **TX Meta Store**: This stores the status of transactions. Before a transaction ID is included in a subtree or block template, the Block Assembly Service checks this database to ensure that the transaction has not

been previously included in another subtree / block.

5. **Subtrees**: Completed subtrees are announced on the network, so other nodes can incorporate them into their blocks.
6. **Hashers / Miners**: After the block template (including all eligible subtrees of transactions) is prepared, it is sent to entities called Miners, which are responsible for performing the computational work (hashing) needed to find a valid block.
7. **Announcement**: All new subtrees that are completed are announced on the IPV6 multicast block subtree network address(es), or any other alternative broadcast service.

4.4. Miner

The Miner (also called Hasher) service is responsible for mining blocks. It solves a hashing Proof of Work for the current set of transactions and subtrees on behalf of a Block Assembly Service, consistently with the Blockchain Whitepaper rules.

Upon successful mining of a block, the miner is rewarded with a block reward (newly minted coins) and the transaction fees from the transactions included in the block.

Challenges and Considerations:

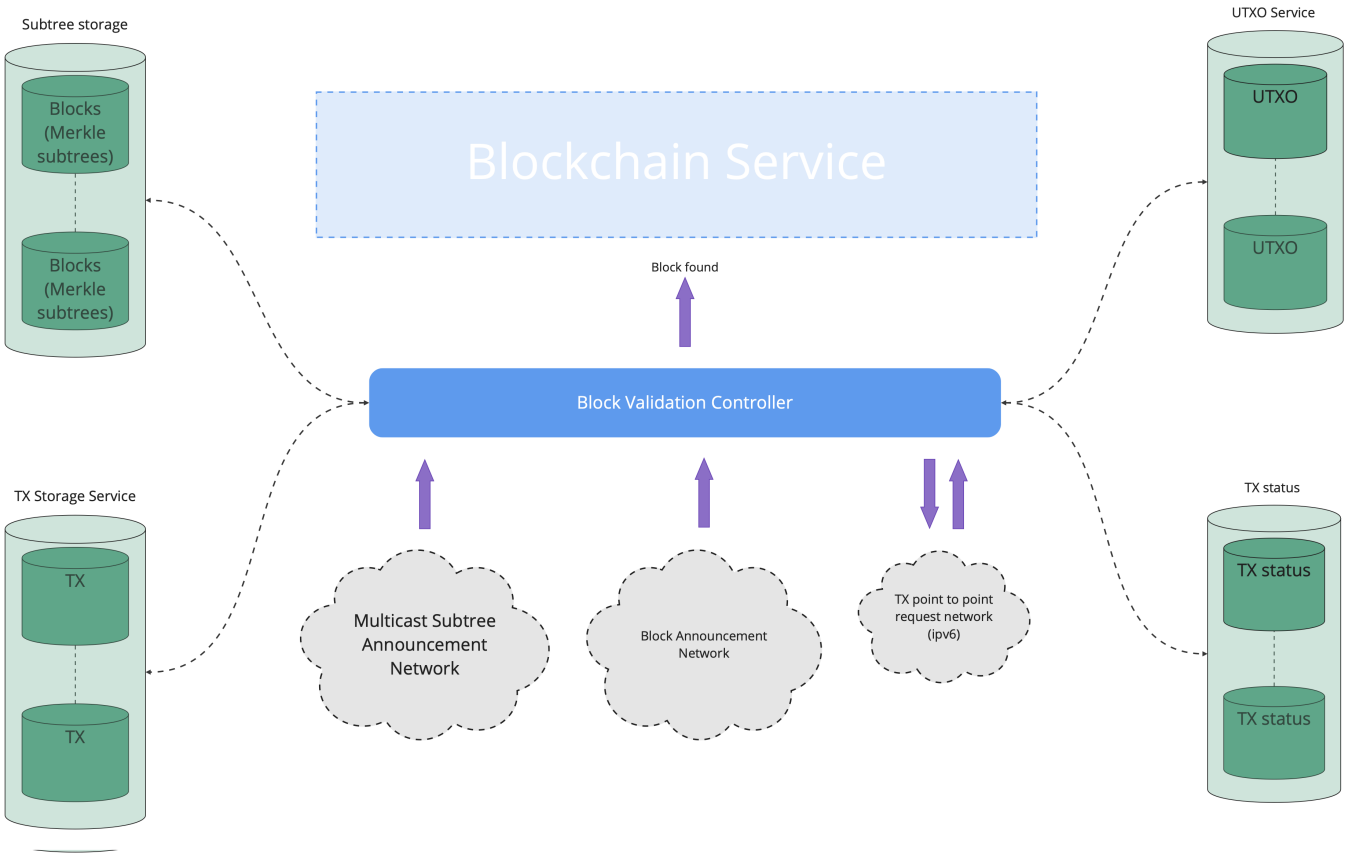
- If two miners solve a block at roughly the same time, there could be a temporary fork in the blockchain. The network resolves this by choosing the chain with the most accumulated work (typically the longest chain).
- Miners must handle orphaned blocks and transactions: if another chain becomes longer, transactions from the orphaned blocks are tracked to be mined in future blocks.

4.5. Subtree and Block Validation

The Block Validation Service is a critical component designed to ensure the integrity and consistency of the blockchain.

Its responsibilities can be broadly categorized into two main areas: **SubTree validation** and **Block validation**.

In addition, the Block Validation Service plays a key role in maintaining the Unspent Transaction Outputs (UTXO) set, which is essential for determining the ownership of coins.



4.5.1. Service Components and Dependencies:

1. **Block Validation Controller:** Acts as the orchestrator for the block validation process. It reacts to new subtrees or blocks being found by either this node or other nodes in the network. The service is tasked with validating the individual subtrees of a block, and later on validating the block that aggregates said subtrees.

2. **Subtree Storage:** Holds the Merkle subtrees, which are partial hashes of the complete block. These are crucial for quickly validating new blocks found by competing miners. These are discarded once they are no longer required (i.e., after a new block is found and the subtrees are no longer needed).

3. **TX Storage Service:** Maintains a record of all created transactions, including those that have not yet been confirmed and added to a block on the blockchain.

4. **UTXO Service:** Manages the list of all unspent transaction outputs, which represent potential inputs for new transactions.

5. **TX Meta Store:** Keeps track of the validation status of individual transactions.

6. **Multicast Subtree Announcement Network:** This network's role is to broadcast Merkle subtree announcements to the network. Multicast allows for efficient distribution of information to nodes interested in receiving these announcements.

7. **TX Point to Point Request Network (IPv6):** This is utilized to request the full transaction data for transactions that are not "blessed."

4.5.2. SubTree Validation Details:

- **Real-Time Validation**: As subtrees are received, which may occur as frequently as every second, the Block Validation service immediately checks their integrity. This involves verifying that each transaction ID listed within a subTree is valid and that the corresponding transactions exist and are themselves valid within the network's consensus rules. If a transaction is missing, we must request it from the miner that sent the subtree. If a transaction is invalid, the subtree is rejected.

- **UTXO Validation**: Part of the validation process includes ensuring that the transactions within a subTree correctly reference and spend UTXOs.

- **Efficiency and Scalability**: By validating subtrees in real-time, the system can efficiently manage a high throughput of transactions, reducing bottlenecks that would otherwise arise during block validation.

- **Handling Unvalidated Transactions**: If a subtree contains a transaction that hasn't been previously validated or "blessed," the Validation service must retrieve and validate that transaction. If the transaction is valid, it is added to the block assembly process. If it is invalid due to issues like missing inputs, the transaction and its subtree are not accepted ("blessed").

4.5.3. Block Validation Details:

- **Top Merkle Tree Validation**: When a new block is announced, the Validator service doesn't need to validate all individual transactions, as the subtrees have already been validated. Instead, it can quickly validate the block by checking the top part of the Merkle tree composed of the hashes of the subtrees. This greatly reduces the amount of data that needs to be processed during block validation.

- **Merkle Subtree Store**: This is a dedicated storage component within the Validator service that holds the subtrees. It retains the subtrees until a new block is found and integrated into the blockchain, after which the subtrees are no longer needed and can be discarded. This storage ensures that subtrees are readily available for block validation and helps in maintaining the continuity of the validation process.

4.5.4. Overall Block and SubTree Validation Process

The validation process is continuous and iterative, designed to maintain the blockchain's integrity and support high transaction throughput:

1. **Transaction and SubTree Receipt**: Transactions are collected and grouped into subtrees, each representing up to 1 million transaction IDs.

2. **SubTree Validation**: As subtrees are broadcast, the Validator service validates them in real-time.
    - If a subtree contains a transaction that has not been previously validated, the Validator service retrieves the full transaction data and validates it.
    - If the Validator service successfully validates a subtree, it is "blessed," indicating approval. If a subtree (or a transaction within the subtree) fails validation, the subtree is rejected.

3. **Merkle Subtree Storage**: Validated subtrees are stored in the Merkle subtree Store.

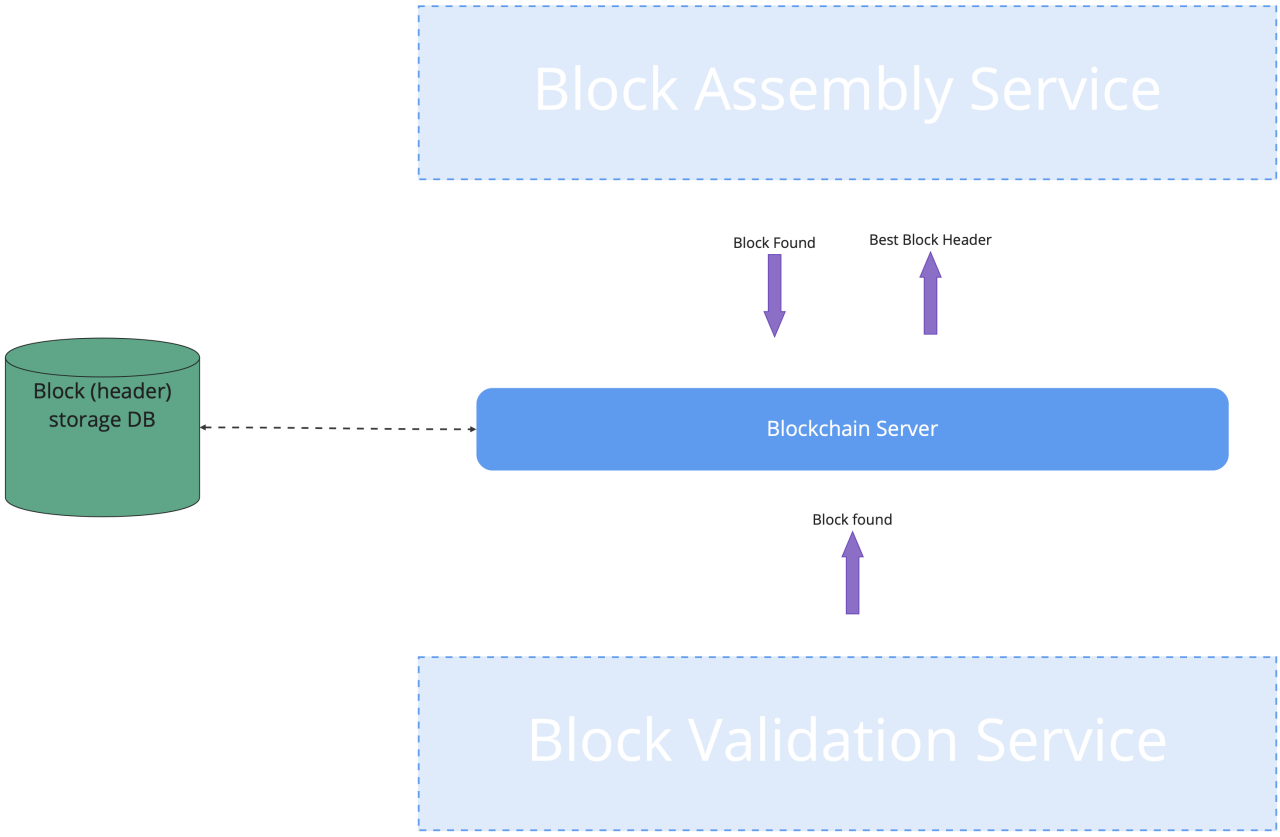
4. **Block Assembly and Propagation**: When a new block is discovered by a miner, it's announced to the network, including only the top-level Merkle hashes of the subtrees.

5. **Block Validation by Validator Service**: The Validator service uses the stored subtrees to validate the newly announced block quickly. This is done by deriving the announced top-level Merkle hashes from the hashes of the subtrees in the Merkle subtree Store.

6. **Blockchain Update**: Once the Validator service validates a block, it propagates this block to the Blockchain service, ensuring that the blockchain remains up-to-date and consistent across all nodes.

4.6. Blockchain Service

The service is reponsible for managing block updates and adding them to the blockchain maintained by the node. The blocks can be received from other nodes or mined by the node itself. Blocks mined by the node are broadcast to other nodes via the Blockchain Service.



Here is an explanation of the process:

- This service manages block headers and lists of subtrees (segments of the blockchain) in each block.
- All blocks are recorded in the blockchain databases, including orphaned blocks (blocks that were not accepted into the main chain).

- Other services can request and store blocks with the service, for purposes like analyzing blockchain data or validating the chain's integrity.

- The service also provides information about the blockchain's current state, such as the best block header (the header of the most recently accepted block, which is critical for mining new blocks) and the current difficulty (a measure of how hard it is to find a new block, which adjusts to keep block discovery times consistent).

Here is an explanation of the components and the process:

1. **Block (header) storage DB**: This is a database that stores block headers. In blockchain systems, a block header is part of a block that includes metadata such as the previous block's hash, the timestamp, the nonce used for mining, and the Merkle tree root.

2. **Blockchain Server**: It acts as the central node that interfaces with the block storage database. It is responsible for processing new blocks (Block Found) and identifying the best block header to be used for further operations like mining or appending to the chain.

3. **Block Assembly Service**: This service takes transactions that are waiting to be included in a block and assembles them into new subtrees and blocks. Once a new block is created (Block Found), it sends this block to the blockchain server.

4. **Block Validation Service**: This component is responsible for validating new blocks. It checks if a block complies with the network's consensus rules. After the blockchain server processes a block (Block Found), it will interact with this service to ensure that the block is valid before finalizing it in the blockchain.

Note - The Block model described in the UBSV Data Model section applies to the internal block model within the UBSV node. The blockchain service stores blocks in the standard Bitcoin format.

The system is designed to maintain the blockchain's integrity by ensuring that all blocks are properly assembled, validated, and stored. It enables other services and participants in the network to interact with the blockchain, request data, and understand the current state of the network for further actions like mining.

4.7. Asset Service

The Asset Service acts as an interface ("Front" or "Facade") to various data stores. It deals with several key data elements:

- **Transactions (TX)**.
- **Subtrees**.
- **Blocks and Block Headers**.
- **Unspent Transaction Outputs (UTXO)**.
- **Metadata for a Transaction (TXMeta)**.

The server uses both HTTP and gRPC as communication protocols:

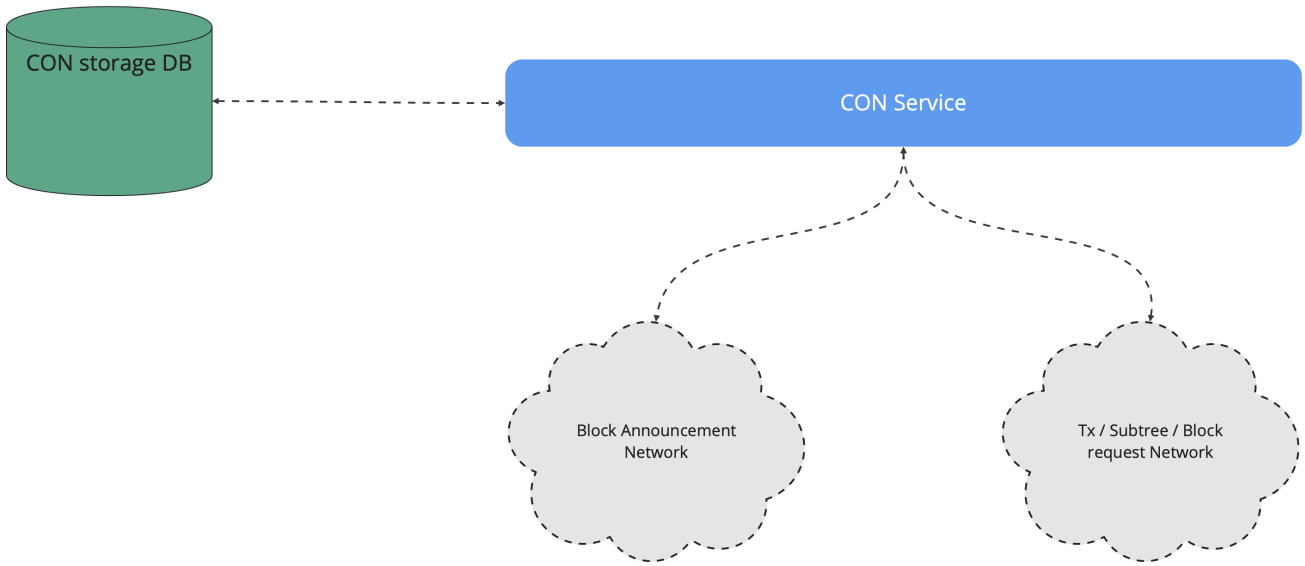
- **HTTP**: A ubiquitous protocol that allows the server to be accessible from the web, enabling other nodes or clients to interact with the server using standard web requests.
- **gRPC**: Allowing for efficient communication between nodes, particularly suited for microservices communication in the UBSV distributed network.

The server being externally accessible implies that it is designed to communicate with other nodes and external clients across the network, to share blockchain data or synchronize states.

The various microservices write directly to the data stores, but the asset service fronts them as a common interface.

4.8. Coinbase Service

The Coinbase Service is designed to monitor the blockchain for new coinbase transactions, record them, track their maturity, and manage the spendability of the rewards miners earn.



In the UBSV context, the "coinbase transaction" is the first transaction in the first subtree of a block and is created by the Block Assembly. This transaction is unique in that it creates new coins from nothing as a reward for the miner's work in processing transactions and securing the network.

The Coinbase Overlay Node (CON) is a specialised service utilized by miners. Its primary function is to monitor all blocks being mined, ensuring accurate tracking of the blocks that have been mined along with their Unspent Transaction Outputs (UTXOs).

This service actively listens for block header notifications, while maintaining a database of all blocks, whether orphaned or not. Upon the announcement of a new block, the service requests the block in the compact subtree format. This format includes the coinbase transaction, which facilitates the processing of any UTXOs linked to that miner.

When a miner intends to spend one of their coins, they need to retrieve the corresponding UTXO from the CON service. Subsequently, they can generate a valid transaction and transmit this through the CON service. This action labels the UTXO as spent.

In essence, the CON service operates as a straightforward Simplified Payment Verification (SPV) overlay node, custom-built to cater to the requirements of miners.

4.9. Bootstrap

The Bootstrap Service helps new nodes find peers in a UBSV network. It allows nodes to register themselves and be notified about other nodes' presence, serving as a discovery service.

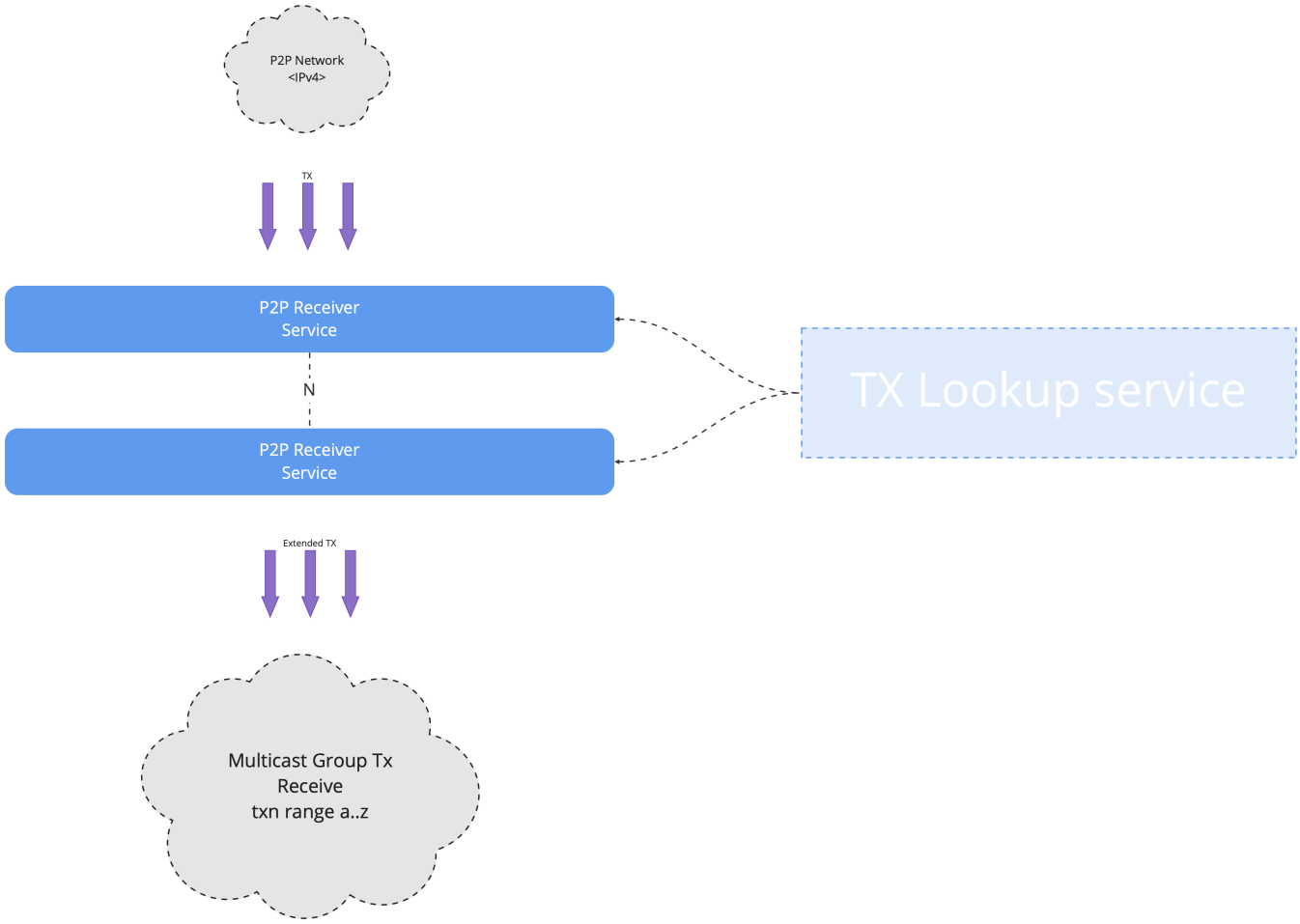
The service manages a set of subscribers and broadcast notifications and keep-alives to them, to keep them updated about network participants.

The current version uses Google's RPC framework for setting up the server and handling stream-based communication.

4.10. P2P Legacy Service

The P2P service is responsible for managing communications between BSV and U-BSV nodes, effectively translating between the historical BSV and the new (UBSV) data abstractions. This makes possible to run historical and UBSV nodes side by side, allowing for a gradual rollout of UBSV.

This legacy P2P network is to be phased out as transaction volumes increase, forcing an eventual migration towards a more scalable and feature-rich system.



This diagram describes the architecture and workflow of the legacy Peer-to-Peer (P2P) service.

Here's the breakdown of the components and their functions:

1. **P2P Network (IPv4)**: This refers to the historical Bitcoin peer-to-peer network using the IPv4 internet protocol.
2. **P2P Receiver Service**: These are the services (1 or more) that receive and send transactions from / to the P2P network.
3. **TX Lookup service**: This service is responsible for looking up previously stored transaction information. It is used to enrich transactions with additional information before they are broadcast to the network.
4. **Multicast Group Tx Receive**: This indicates a multicast setup where transactions are broadcast to multiple nodes simultaneously. This is efficient for disseminating information quickly to many nodes in the network.

4.11. UTXO Store

The UTXO Store service is responsible for tracking spendable UTXOs. These are UTXOs that can be used as inputs in new transactions. The UTXO Store service is primarily used by the Validator service to retrieve

UTXOs when validating transactions. The main purpose of this service is to provide a quick lookup service on behalf of other micro-services (such as the Validator service).

4.12. Transaction Meta Store

The Transaction Meta Store service is responsible for storing and retrieving transaction metadata. This is used by many services, including the Validator and Block Assembly services, to retrieve transaction metadata when validating transactions. The Transaction Meta Store service is also used by the Block Assembly service to retrieve transaction metadata when assembling blocks.

The metadata in scope in this service refers to extra fields of interest during transaction-related processing.

Field	Description
Tx	The actual transaction data
Fee	The fee associated with the transaction
SizeInBytes	The size of the transaction in bytes
FirstSeen	Timestamp of when the transaction was first seen
ParentTxHashes	List of hashes of the transaction's parent transactions

4.13. Banlist Service

Bitcoin is an open public system that anyone can use. While most participants act in good faith, the system needs to protect itself against rogue agents. If a node is breaching the network consensus rules (a "rogue" node), it will get banned.

For example, any node trying to introduce a double spend will be banned. Equally, not pre-announcing a significant % of the subtrees before the block is found, or not broadcasting the block after it is found, will play against the consensus rules and will get a node banned.

Once a node is banned, any transaction, subtree or block coming from that node will be rejected.